Abaixo está um plano completo e estruturado, pensado como um Arquiteto de Testes + QA Sênior, para você **criar, priorizar e documentar cenários em Gherkin** para o SauceDemo — mesmo sem acesso a código.

---

# ☑ 1. PLANO DETALHADO (EXECUTÁVEL MANUALMENTE)

## 🔎 Etapa 1 — Levantamento Funcional Exploratório

**Objetivo:** identificar funcionalidades reais do sistema (base para cenários)

### Passos:

1. Acesse o sistema:
   👉 Abrir Sauce Demo
2. Explore manualmente os fluxos principais:
   - Login
   - Listagem de produtos
   - Carrinho
   - Checkout
   - Logout
   - Ordenação
   - Reset app state
3. Utilize diferentes usuários:

```
standard_user
locked_out_user
problem_user
performance_glitch_user
```

1. Documente manualmente:
   - Passos executados
   - Resultado esperado
   - Comportamento real

---

## 🧠 Etapa 2 — Identificação de Fluxos Críticos

Classifique em três níveis:

### Alta prioridade (core business)

- Login
- Compra completa (end-to-end)
- Carrinho

### Média prioridade

- Ordenação de produtos
- Remoção de itens
- Persistência de sessão

### Baixa prioridade

- Layout
- Performance percebida
- Mensagens visuais não críticas

---

## Etapa 3 — Definição de Critérios de Cobertura

Para cada funcionalidade, cubra:

| Tipo de cenário | Exemplo |
|---|---|
| **Caminho feliz** | Compra completa |
| **Negativo** | Login inválido |
| **Borda** | Carrinho vazio |
| **Alternativo** | Remover item e continuar comprando |

---

## Etapa 4 — Estruturação do Gherkin

### Padrão obrigatório:

```
Funcionalidade: <Nome da funcionalidade>

  Contexto:
    Dado que ...

  Cenário: <Título claro>
    Dado que ...
    Quando ...
    Então ...
```

# 2. CENÁRIOS GHERKIN (PRIORIZADOS)

## ALTA PRIORIDADE

### 1. Login

```
Funcionalidade: Login de usuário

  Cenário: Login com sucesso
    Dado que o usuário acessa a página de login
    Quando informa usuário "standard_user" e senha "secret_sauce"
    E clica no botão de login
    Então deve ser redirecionado para a página de produtos

  Cenário: Login com credenciais inválidas
     Dado que o usuário acessa a página de login
     Quando informa usuário inválido e senha inválida
     E clica no botão de login
     Então deve visualizar mensagem de erro

   Cenário: Usuário bloqueado
     Dado que o usuário acessa a página de login
     Quando informa usuário "locked_out_user" e senha "secret_sauce"
     E clica no botão de login
     Então deve visualizar mensagem de usuário bloqueado
```

### 2. Compra Completa (E2E)

```
Funcionalidade: Processo de compra

  Cenário: Compra de produto com sucesso
    Dado que o usuário está logado
    Quando adiciona um produto ao carrinho
    E acessa o carrinho
    E inicia o checkout
    E informa nome, sobrenome e CEP
```

```
9       E finaliza a compra
10      Então deve visualizar mensagem de confirmação
```

### 3. Carrinho

```
1   Funcionalidade: Carrinho de compras
2
3     Cenário: Adicionar item ao carrinho
4       Dado que o usuário está na página de produtos
5       Quando adiciona um produto ao carrinho
6       Então o carrinho deve mostrar 1 item
7
8     Cenário: Remover item do carrinho
9       Dado que o usuário adicionou um item ao carrinho
10      Quando remove o item
11      Então o carrinho deve estar vazio
```

## MÉDIA PRIORIDADE

### 4. Ordenação de Produtos

```
1   Funcionalidade: Ordenação de produtos
2
3     Cenário: Ordenar produtos por preço crescente
4       Dado que o usuário está na página de produtos
5       Quando seleciona ordenar por preço crescente
6       Então os produtos devem ser exibidos do menor para o maior
preço
7
8     Cenário: Ordenar produtos por nome
9       Dado que o usuário está na página de produtos
10      Quando seleciona ordenar por nome de A a Z
11      Então os produtos devem ser exibidos em ordem alfabética
```

### 5. Continuidade de Compra

```
1   Funcionalidade: Continuação de compras
2
3     Cenário: Continuar comprando após adicionar item
```

```
4       Dado que o usuário adicionou um item ao carrinho
5       Quando volta para a lista de produtos
6       Então deve continuar visualizando os produtos disponíveis
```

### 6. Checkout - Validações

```
1   Funcionalidade: Validação do checkout
2
3     Cenário: Checkout sem preencher dados
4       Dado que o usuário iniciou o checkout
5       Quando não informa os dados obrigatórios
6       E tenta continuar
7       Então deve visualizar mensagem de erro
8
9     Cenário: Checkout parcial
10      Dado que o usuário iniciou o checkout
11      Quando preenche apenas parte dos dados
12      Então deve visualizar erro de validação
```

## BAIXA PRIORIDADE

### 7. Logout

```
1   Funcionalidade: Logout
2
3     Cenário: Logout com sucesso
4       Dado que o usuário está logado
5       Quando clica no botão de logout
6       Então deve ser redirecionado para a página de login
```

### 8. Reset da aplicação

```
1   Funcionalidade: Resetar estado da aplicação
2
3     Cenário: Resetar carrinho
4       Dado que o usuário possui itens no carrinho
5       Quando reseta o estado da aplicação
6       Então o carrinho deve ficar vazio
```

### 9. Carrinho vazio

```
Funcionalidade: Carrinho vazio

  Cenário: Acessar carrinho sem itens
    Dado que o usuário não possui itens no carrinho
    Quando acessa o carrinho
    Então deve visualizar que o carrinho está vazio
```

---

## 📊 3. SUMÁRIO (COMO O PLANO FOI DESENVOLVIDO)

O plano foi estruturado com base em **estratégia de testes orientada a risco + cobertura funcional**, seguindo estes princípios:

### ✓ Abordagem usada:

1. **Análise funcional exploratória** do SauceDemo
2. **Identificação dos fluxos críticos (login + compra)**
3. Aplicação de **pirâmide de valor de automação**
4. Estruturação em **BDD (Gherkin)** para integração com:
    - Pytest
    - Guará Framework
    - Page Objects

### ✓ Resultado:

- Cenários prontos para automação
- Priorização baseada em impacto de negócio
- Cobertura completa (happy path + negativos + borda)

---

## 🚀 4. BOAS PRÁTICAS (MELHORIAS FUTURAS)

### 🧩 Organização

- Separar arquivos .feature por domínio:

```
login.feature
cart.feature
checkout.feature
products.feature
```

## ⚙️ Para automação futura (Guará + Selenium)

- Usar Page Objects para:
  - LoginPage
  - InventoryPage
  - CartPage
  - CheckoutPage
- Criar Transactions:
  - perform_login()
  - add_to_cart()
  - checkout()

---

## 🧪 Estratégia de dados

- Centralizar credenciais em fixtures
- Usar massa de teste configurável

---

## 🖌️ Manutenibilidade

- Evitar steps muito técnicos
- Focar em linguagem de negócio

✅ Exemplo correto:

```
Quando adiciona um produto ao carrinho
```

❌ Evitar:

```
Quando clica no botão com id btn-add-to-cart
```

---

## 📐 Expansão futura

Adicionar:

- Testes de performance (usuário performance_glitch_user)
- Testes visuais (comparação de layout)
- Testes cross-browser

---

# ☑ CONCLUSÃO

Você agora tem:

- Um **plano completo executável**
- Cenários **Gherkin prontos para uso**
- Organização pronta para **Guará + Pytest + Selenium**

---

Se quiser, posso dar o próximo passo e já gerar: ✅ Estrutura de pastas do projeto
✅ Esqueleto de automação com Page Objects + Pytest
✅ Steps prontos para Behave / Guará

Só me diga 👍